

Melodia



DOCUMENTATION TECHNIQUE



Version 1.0 — 2025-2026

Metais Romain · Berlot Aurélien · Guillard Noah · Marlier Thibaud

# Table des matières

|                                                  |    |
|--------------------------------------------------|----|
| Table des matières                               | 2  |
| 1. Introduction                                  | 4  |
| 1.1 Objectif du document                         | 4  |
| 1.2 Présentation du projet                       | 4  |
| 1.3 Périmètre du projet                          | 4  |
| 1.4 Fonctionnalités principales                  | 4  |
| 2. Architecture générale                         | 5  |
| 2.1 Vue d'ensemble                               | 5  |
| 2.2 Description des composants                   | 5  |
| 2.3 Flux de communication                        | 6  |
| 2.4 Stratégie de cache                           | 6  |
| 3. Choix technologiques                          | 7  |
| 3.1 API tierce musicale                          | 7  |
| 3.2 Backend                                      | 7  |
| 3.3 Base de données                              | 7  |
| 3.4 Client Web                                   | 7  |
| 3.5 Client Mobile                                | 8  |
| 3.6 Infrastructure & outillage                   | 8  |
| 4. Backend — API REST                            | 8  |
| 4.1 Structure du projet                          | 8  |
| 4.2 Middlewares                                  | 8  |
| 4.3 Gestion des erreurs                          | 9  |
| 5. Modèle de données                             | 10 |
| 5.1 Schéma relationnel (ERD)                     | 10 |
| 5.2 Description des entités principales          | 10 |
| 6. API Serveur — Endpoints                       | 13 |
| 6.1 Utilisateurs — /users                        | 13 |
| 6.2 Critiques — /reviews                         | 13 |
| 6.3 Commentaires — /review-comments              | 14 |
| 6.4 Médias & statuts — /medias                   | 15 |
| 6.5 Playlists — /playlists & /playlist-items     | 15 |
| 6.6 Social — /follows, /conversations, /messages | 15 |
| 6.7 API externe Last.fm — /api                   | 16 |
| 6.8 OAuth — /api/oauth                           | 16 |
| 7. Authentification & Sécurité                   | 17 |
| 7.1 Authentification locale (JWT)                | 17 |
| 7.2 OAuth 2.0 (Google & Discord)                 | 17 |
| 7.3 Contrôle d'accès basé sur les rôles (RBAC)   | 18 |

|                                                         |    |
|---------------------------------------------------------|----|
| 7.4 Sécurité applicative                                | 18 |
| 8. Temps réel & Notifications                           | 18 |
| 8.1 Architecture Socket.io                              | 18 |
| 8.2 Notifications push (mobile)                         | 18 |
| 9. Intégration Last.fm                                  | 19 |
| 10. Client Web                                          | 20 |
| 10.1 Architecture React                                 | 20 |
| 10.2 Pages et routing                                   | 20 |
| 10.3 Composants principaux                              | 21 |
| 11. Application Mobile                                  | 21 |
| 11.1 Architecture Expo                                  | 21 |
| 11.2 Navigation et écrans                               | 21 |
| 11.3 Système de thèmes (Dark / Light)                   | 22 |
| 12. Diagrammes UML                                      | 23 |
| 12.1 Diagramme de cas d'utilisation                     | 23 |
| 12.2 Diagramme de séquence – Recherche musicale         | 24 |
| 12.3 Diagramme de séquence – Authentification OAuth2    | 24 |
| 12.4 Diagramme de séquence – Publication d'une critique | 25 |
| 13. Infrastructure & Déploiement                        | 26 |
| 13.1 Prérequis                                          | 26 |
| 13.2 Obtention de la clé API Last.fm                    | 26 |
| 13.3 Configuration de l'environnement                   | 26 |
| 13.4 Lancement avec Docker Compose                      | 27 |
| 13.5 Architecture Docker Compose                        | 27 |
| 13.6 Domaines et réseau (production)                    | 27 |
| 13.7 Commandes de développement                         | 28 |
| 14. Qualité du code & conventions                       | 28 |
| 14.1 Structure des dépôts                               | 28 |
| 14.2 Conventions de nommage                             | 28 |
| 14.3 Linting & formatage                                | 28 |
| 14.4 Tests                                              | 28 |
| 15. Gestion de projet & contribution                    | 29 |
| 15.1 Dépôt Git                                          | 29 |
| 15.2 Répartition des tâches                             | 29 |
| 16. Annexes                                             | 30 |
| 16.1 Références                                         | 30 |
| 16.2 Exemple de réponse API                             | 30 |

# 1. Introduction

## 1.1 Objectif du document

Ce document est la documentation technique officielle du projet Melodia. Il est destiné aux développeurs, aux évaluateurs et à toute personne souhaitant comprendre, installer, déployer ou maintenir l'application. Le projet a été réalisé par une équipe de quatre étudiants dans le cadre du projet de fin d'année.

## 1.2 Présentation du projet

Melodia est un réseau social centré sur la musique. L'application permet aux utilisateurs de découvrir des albums et artistes, de gérer leur collection musicale personnelle, d'échanger des avis et de discuter en temps réel avec une communauté de passionnés. L'objectif est de rassembler les mélomanes au sein d'une seule application. Le projet s'organise en monorepo comprenant trois applications distinctes – un backend API REST, un client web et une application mobile – toutes communicantes via des interfaces communes.

## 1.3 Périmètre du projet

Le périmètre couvre l'ensemble des composants nécessaires au fonctionnement de l'application :

- l'interface utilisateur accessible via le web et les appareils mobiles ;
- la base de données assurant le stockage persistant des informations ;
- l'intégration d'un service externe enrichissant les fonctionnalités liées à la gestion et à la consultation des données musicales.

L'API externe retenue est Last.fm, choisie pour la richesse de son catalogue et de ses métadonnées, et pour son adéquation avec les besoins de Melodia.

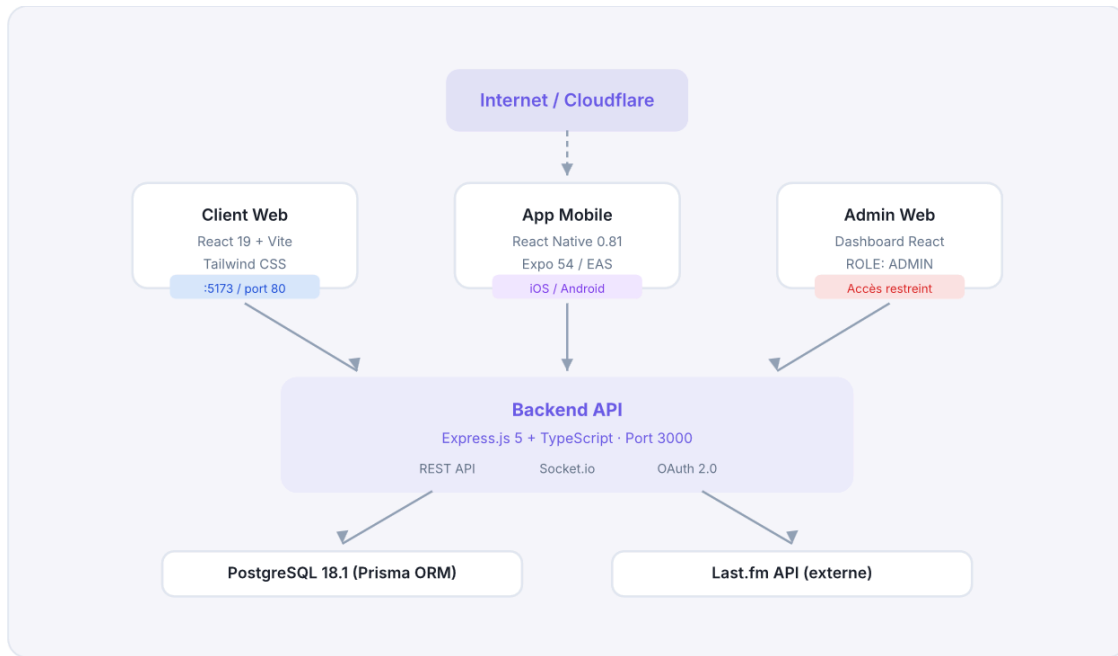
## 1.4 Fonctionnalités principales

- Musique & découverte : recherche d'albums et d'artistes via l'API Last.fm. Consultation de fiches détaillées avec tags, pistes et albums similaires.
- Critiques & notes : rédaction de critiques d'albums avec notation de 1 à 5. Likes sur les critiques, commentaires threadés (réponses imbriquées).
- Bibliothèque personnelle : création de playlists publiques ou privées. Statut par album : Écouté, À écouter, Favori, Pas aimé.
- Social : abonnements (following / followers), fil d'activité, messagerie directe en temps réel.
- Notifications : notifications in-app et push (mobile) pour les nouveaux messages, likes, abonnements et recommandations.
- Modération : signalement de contenu (critiques, profils, commentaires), tableau de bord administrateur, système de bannissement.

## 2. Architecture générale

### 2.1 Vue d'ensemble

L'application repose sur une architecture trois-tiers stricte. Les clients, qu'ils soient web ou mobile, ne communiquent jamais directement avec l'API tierce musicale : toute la logique métier est centralisée dans le backend. L'application est déployée via Docker sur un serveur unique, exposé par Cloudflare Tunnel.



### 2.2 Description des composants

| Couche           | Technologie             | Version    | Rôle                                  |
|------------------|-------------------------|------------|---------------------------------------|
| Backend          | Express.js + TypeScript | 5.0 / 5.9  | API REST, WebSocket, OAuth            |
| ORM              | Prisma                  | 7.3        | Accès base de données typé (PrismaPg) |
| Base de données  | PostgreSQL              | 18.1       | Stockage persistant relationnel       |
| Temps réel       | Socket.io               | 4.8        | Messagerie bidirectionnelle           |
| Authentification | JWT + Passport.js       | 9.0 / 0.7  | Auth locale + OAuth Google/Discord    |
| Client web       | React + Vite            | 19.2 / 7.2 | SPA, routage client                   |
| Styles web       | Tailwind CSS            | 4.1        | Utility-first CSS                     |
| Mobile           | React Native + Expo     | 0.81 / 54  | iOS & Android natif                   |
| Routing mobile   | Expo Router             | 6.0        | File-based routing                    |

|                    |                   |     |                            |
|--------------------|-------------------|-----|----------------------------|
| Push notifications | Expo Server SDK   | 6.1 | Notifications mobiles      |
| Conteneurs         | Docker / Compose  | —   | Orchestration des services |
| Réseau             | Cloudflare Tunnel | —   | HTTPS public sans IP fixe  |

## 2.3 Flux de communication

Les clients (web et mobile) communiquent avec le backend via des requêtes HTTP REST et via des WebSockets (Socket.io) pour les fonctionnalités temps réel. L'authentification s'effectue soit localement via JWT, soit via OAuth 2.0 (Google et Discord). Le backend agit comme intermédiaire unique avec l'API externe Last.fm pour récupérer les données musicales, évitant toute communication directe entre les clients et l'API tierce.

## 2.4 Stratégie de cache

Les données récupérées depuis l'API Last.fm sont mises en cache directement dans la base PostgreSQL, au sein de la table Medias. Un mécanisme d'expiration est géré via le champ `expires_at` (TTL de 60 minutes) afin d'invalider le cache de manière autonome et de réduire le nombre d'appels externes. En cas de dépassement des quotas de l'API Last.fm, les requêtes échouent silencieusement et le cache existant est retourné si disponible, afin d'assurer la continuité du service.

## 3. Choix technologiques

Cette section justifie chaque choix de technologie retenu pour le projet.

### 3.1 API tierce musicale

L'API retenue est celle de Last.fm, une API ouverte et stable de longue date, offrant des métadonnées riches : noms d'artistes, pochettes d'albums, années de sortie, tags, pistes, albums et artistes similaires. L'API de Spotify a également été étudiée mais n'a pas été retenue : des incertitudes pesaient sur la garantie d'accès à tous les endpoints clés, ce qui aurait restreint les possibilités fonctionnelles de l'application.

### 3.2 Backend

- Framework : Express.js 5 + TypeScript (5.0 / 5.9).
- Justification : permet une architecture 3-tiers robuste via un pattern Controller / Service / Mapper, gérant à la fois l'API REST, les WebSockets et l'OAuth dans une base de code unique et typée.
- Format d'API : API REST + Socket.io pour la messagerie bidirectionnelle.
- Authentification : JWT + Passport.js (stratégies locale, Google et Discord).

### 3.3 Base de données

- SGBD : PostgreSQL 18.1.
- Justification : assure un stockage persistant relationnel optimal pour les liens complexes du domaine (followers, playlists, critiques, conversations).
- ORM : Prisma 7.3 avec l'adaptateur PrismaPg, pour un accès typé et des migrations versionnées.

### 3.4 Client Web

Le choix de React s'est imposé rapidement compte tenu de l'ampleur du site à développer. Ses composants réutilisables facilitent l'organisation du code et la maintenance ; sa documentation abondante et sa prise en main accessible en font une solution adaptée.

Principales bibliothèques :

- react-router-dom — navigation entre les pages.
- Context API — gestion de l'état global (authentification, socket).
- Lucide React — jeu d'icônes modernes et personnalisables.
- react-i18next — internationalisation de l'application.

Framework : React 19 + Vite 7.2 · Styles : Tailwind CSS 4.1.

## 3.5 Client Mobile

L'utilisation de React Native avec Expo permet de développer une application compatible Android et iOS à partir d'un seul code source. Expo simplifie la configuration du projet et accélère les phases de développement et de test, ce qui a permis de gagner du temps. La distribution se fait via EAS (Expo Application Services).

## 3.6 Infrastructure & outillage

| Outil                   | Usage                 | Justification                                     |
|-------------------------|-----------------------|---------------------------------------------------|
| Docker / Docker Compose | Conteneurisation      | Déploiement reproductible, isolation des services |
| Cloudflare Tunnel       | Exposition réseau     | HTTPS public sans IP fixe ni ouverture de ports   |
| Git + GitHub            | Versionnement         | Historique, branches, revue de code               |
| Prisma Migrate          | Migrations BDD        | Schéma versionné et reproductible                 |
| nodemon + tsx           | Développement backend | Rechargement à chaud du serveur TypeScript        |

## 4. Backend – API REST

### 4.1 Structure du projet

```
backend/
├── app.ts                # Express app (middlewares, routes, gestion erreurs)
├── bin/www.ts            # Point d'entrée HTTP + initialisation Socket.io
├── config/
│   ├── database.ts      # Connexion Prisma + adaptateur PrismaPg
│   └── passport.ts      # Stratégies OAuth (Google, Discord)
├── middlewares/         # Auth, rôles, propriété des ressources
├── modules/
│   ├── db/              # Modules métier (controller + service + mapper/helper)
│   ├── external_api/    # Intégration Last.fm (albums, artists, tracks, tags, search)
│   ├── oauth/           # Contrôleur OAuth
│   └── web_socket/      # Serveur Socket.io
├── routes/              # Routes REST (db, api, oauth)
├── mappers/             # Transformation entité Prisma → DTO
├── types/               # Interfaces TypeScript (DTO)
├── utils/               # Erreurs, helpers, validateurs
└── prisma/schema.prisma # Définition du schéma BDD
```

Pattern architectural – Chaque module suit un enchaînement Contrôleur → Service → Mapper : le contrôleur gère la requête HTTP, le service contient la logique métier et l'accès Prisma, le mapper transforme les entités Prisma en DTOs exposés à l'API.



## 4.2 Middlewares

| Middleware                | Fichier                                  | Description                                                                                                   |
|---------------------------|------------------------------------------|---------------------------------------------------------------------------------------------------------------|
| authGuard                 | middlewares/auth.ts                      | Vérifie le Bearer JWT dans le header Authorization et attache req.user. Variante optionalAuthGuard tolérante. |
| checkAdmin                | middlewares/checkAdmin.ts                | Vérifie que req.user.role === "ADMIN".                                                                        |
| checkResourceOwnerOrAdmin | middlewares/checkResourceOwnerOrAdmin.ts | Autorise le propriétaire de la ressource ou un admin.                                                         |
| checkPlaylistOwner        | middlewares/checkPlaylistOwner.ts        | Vérifie que la playlist appartient à l'utilisateur.                                                           |
| CORS                      | app.ts                                   | Origines : localhost:5173, melodia.cloud, www.melodia.cloud.                                                  |
| express.json              | app.ts                                   | Parsing JSON, limite 10 Mo (images base64).                                                                   |
| morgan                    | app.ts                                   | Logging HTTP en mode développement.                                                                           |

## 4.3 Gestion des erreurs

Un middleware centralisé dans app.ts intercepte toutes les erreurs et retourne une réponse JSON normalisée :

```
{
  "error": 400,
  "message": "Email or username already in use",
  "stack": "...",
  "details": {}
}
```

Classes d'erreurs personnalisées dans utils/errors.ts :

| Classe       | Code HTTP | Usage                                  |
|--------------|-----------|----------------------------------------|
| BadRequest   | 400       | Données invalides ou manquantes        |
| Unauthorized | 401       | Token manquant, invalide ou expiré     |
| Forbidden    | 403       | Droits insuffisants                    |
| NotFound     | 404       | Ressource inexistante                  |
| ApiError     | variable  | Classe de base pour les erreurs métier |

## 5. Modèle de données

Le projet utilise PostgreSQL 18.1 avec l'ORM Prisma 7.3 (adaptateur PrismaPg). Le schéma est défini dans backend/prisma/schema.prisma.

### 5.1 Schéma relationnel (ERD)

Relations principales :

- Users 1—N Reviews, Playlists, Notifications, Activities, Messages.
- Users N—N Users via Follows (abonnements) et via Conversations (paires uniques).
- Medias 1—N Reviews, PlaylistItems, UserMediaStatus.
- Reviews 1—N ReviewComments (auto-référencés par parent\_id) et ReviewLikes.
- Reports référence un profil, une critique ou un commentaire signalé.

### 5.2 Description des entités principales

5.2.1 Utilisateur — Users. Entité centrale du système. Supporte l'authentification locale (mot de passe haché bcrypt) et OAuth (provider, provider\_id). La photo de profil est stockée en binaire (Bytes PostgreSQL) et exposée en base64 via les mappers.

| Champ           | Type         | Contrainte       | Description                           |
|-----------------|--------------|------------------|---------------------------------------|
| id              | UUID         | PK               | Identifiant unique                    |
| username        | VARCHAR(25)  | UNIQUE, NOT NULL | Nom d'affichage public                |
| email           | VARCHAR(320) | UNIQUE, NOT NULL | Adresse de connexion                  |
| password        | String       | NULLABLE         | Hash bcrypt (null si compte OAuth)    |
| role            | ENUM         | NOT NULL         | BASIC   ADMIN (défaut BASIC)          |
| provider        | ENUM         | NULLABLE         | LOCAL   GOOGLE   DISCORD   FACEBOOK   |
| provider_id     | String       | NULLABLE         | Identifiant chez le fournisseur OAuth |
| favorite_band   | String       | NULLABLE         | Artiste favori                        |
| biography       | VARCHAR(255) | NULLABLE         | Biographie                            |
| phone_number    | VARCHAR(15)  | NULLABLE         | Numéro de téléphone                   |
| profile_picture | Bytes        | NULLABLE         | Avatar (binaire, exposé en base64)    |

|                         |           |             |                             |
|-------------------------|-----------|-------------|-----------------------------|
| has_notifications       | Boolean   | défaut true | Préférence de notifications |
| expo_push_token         | String    | NULLABLE    | Token Expo pour push mobile |
| created_at / updated_at | TIMESTAMP | NOT NULL    | Horodatage                  |

Contrainte d'unicité composite : (provider, provider\_id).

5.2.2 Œuvre musicale — Medias. Cache des métadonnées d'albums provenant de Last.fm. Le champ content est un JSON brut retourné par l'API externe. Le champ expires\_at permet l'invalidation du cache. La note globale rating est mise à jour à chaque nouvelle critique. Cette entité sert uniquement de cache local et n'est jamais alimentée manuellement par les utilisateurs.

| Champ                   | Type      | Contrainte | Description                     |
|-------------------------|-----------|------------|---------------------------------|
| id                      | UUID      | PK         | Identifiant interne             |
| api_id                  | String    | UNIQUE     | Identifiant Last.fm             |
| content                 | JSON      | NOT NULL   | Métadonnées brutes de l'API     |
| rating                  | Float     | NULLABLE   | Note moyenne agrégée            |
| created_at / expires_at | TIMESTAMP | NOT NULL   | Création et expiration du cache |

5.2.3 Critique — Reviews. Un utilisateur ne peut rédiger qu'une seule critique par album (contrainte unique composite (user\_id, media\_id)).

| Champ                   | Type          | Contrainte | Description          |
|-------------------------|---------------|------------|----------------------|
| id                      | UUID          | PK         | Identifiant          |
| user_id                 | UUID          | FK Users   | Auteur               |
| media_id                | UUID          | FK Medias  | Album concerné       |
| rating                  | Int           | NOT NULL   | Note de 1 à 5        |
| title                   | VARCHAR(100)  | NOT NULL   | Titre de la critique |
| content                 | VARCHAR(1000) | NOT NULL   | Contenu              |
| created_at / updated_at | TIMESTAMP     | NOT NULL   | Horodatage           |

5.2.4 Commentaires & likes — ReviewComments, ReviewLikes, CommentLikes. Les commentaires supportent l'imbrication via parent\_id (auto-référence, suppression en cascade). Les likes sur critiques (ReviewLikes) et commentaires (CommentLikes) utilisent des tables de jointure à clé primaire composite.



### 5.2.5 Playlists – Playlists, PlaylistItems.

| Champ     | Type        | Contrainte      | Description         |
|-----------|-------------|-----------------|---------------------|
| id        | UUID        | PK              | Identifiant         |
| user_id   | UUID        | FK Users        | Propriétaire        |
| name      | VARCHAR(30) | UNIQUE par user | Nom de la playlist  |
| image_url | Bytes       | NULLABLE        | Image de couverture |
| is_public | Boolean     | NOT NULL        | Visibilité          |

PlaylistItems relie une playlist à un média (unique (playlist\_id, media\_id), cascade à la suppression de la playlist).

5.2.6 Statut de bibliothèque – UserMediaStatus. Table de statuts personnels par média, clé primaire composite (user\_id, media\_id). Valeurs possibles : listened, later, favorite, disliked, none.

### 5.2.7 Autres entités.

- Follows – abonnements, clé composite (user\_id, follow\_user\_id).
- Activities – journal des actions pour le fil d’actualité : review\_created, rating\_added, playlist\_added, follow.
- Notifications – messages in-app ciblés avec statut de lecture (is\_read, read\_at). Actions : review\_added, new\_message, like\_added, comment\_added, new\_follow, recommendation.
- Conversations / Messages – messagerie directe entre deux utilisateurs, conversation unique par paire (user1\_id, user2\_id). Statut de lecture is\_read.
- Reports – signalements de type review, profile ou comment, avec reason, reason\_type et is\_checked.
- BannedUsers – utilisateurs bannis (conserve username, email et motif).

## 6. API Serveur – Endpoints

L'API suit les principes REST. L'authentification via JWT (header Authorization: Bearer <token>) est requise sauf mention contraire. Codes HTTP utilisés : 200, 201, 400, 401, 403, 404, 500.

### 6.1 Utilisateurs – /users

| Méthode | Route                    | Auth              | Description                               |
|---------|--------------------------|-------------------|-------------------------------------------|
| POST    | /users/signin            | —                 | Inscription (email / password / username) |
| POST    | /users/login             | —                 | Connexion → retourne un JWT               |
| PATCH   | /users/profile           | JWT               | Mise à jour du profil                     |
| GET     | /users/public/:id        | —                 | Profil public d'un utilisateur            |
| GET     | /users/:id               | JWT + Owner/Admin | Profil complet                            |
| PUT     | /users/:id               | JWT + Owner/Admin | Mise à jour complète                      |
| DELETE  | /users/:id               | JWT + Owner/Admin | Suppression du compte                     |
| GET     | /users                   | JWT + Admin       | Liste de tous les utilisateurs            |
| POST    | /users/update-push-token | JWT               | Enregistrement du token Expo push         |

### 6.2 Critiques – /reviews

| Méthode | Route                   | Auth              | Description                               |
|---------|-------------------------|-------------------|-------------------------------------------|
| POST    | /reviews                | JWT               | Créer une critique (1 max par album/user) |
| GET     | /reviews/:id            | —                 | Détail d'une critique                     |
| PUT     | /reviews/:id            | JWT + Owner       | Modifier une critique                     |
| DELETE  | /reviews/:id            | JWT + Owner/Admin | Supprimer une critique                    |
| GET     | /reviews/media/:mediaId | —                 | Toutes les critiques d'un album           |

|      |                   |     |                              |
|------|-------------------|-----|------------------------------|
| POST | /reviews/:id/like | JWT | Liker / unliker une critique |
|------|-------------------|-----|------------------------------|

### 6.3 Commentaires — /review-comments

| Méthode | Route                             | Auth              | Description                            |
|---------|-----------------------------------|-------------------|----------------------------------------|
| POST    | /review-comments                  | JWT               | Poster un commentaire (avec parent_id) |
| GET     | /review-comments/review/:reviewId | —                 | Commentaires d'une critique (threadés) |
| DELETE  | /review-comments/:id              | JWT + Owner/Admin | Supprimer un commentaire               |
| POST    | /review-comments/:id/like         | JWT               | Liker / unliker un commentaire         |

## 6.4 Médias & statuts – /medias

| Méthode | Route               | Auth | Description                                 |
|---------|---------------------|------|---------------------------------------------|
| GET     | /medias/:apild      | —    | Info d'un média (cache BDD ou Last.fm)      |
| POST    | /medias/:id/status  | JWT  | Définir le statut d'un album                |
| GET     | /medias/user/status | JWT  | Statuts de tous les albums de l'utilisateur |

## 6.5 Playlists – /playlists & /playlist-items

| Méthode | Route                   | Auth              | Description                     |
|---------|-------------------------|-------------------|---------------------------------|
| POST    | /playlists              | JWT               | Créer une playlist              |
| GET     | /playlists/user/:userId | —                 | Playlists d'un utilisateur      |
| PUT     | /playlists/:id          | JWT + Owner       | Modifier une playlist           |
| DELETE  | /playlists/:id          | JWT + Owner/Admin | Supprimer une playlist          |
| POST    | /playlist-items         | JWT + Owner       | Ajouter un média à une playlist |
| DELETE  | /playlist-items/:id     | JWT + Owner       | Retirer un média d'une playlist |

## 6.6 Social – /follows, /conversations, /messages

| Méthode | Route                      | Auth | Description                                |
|---------|----------------------------|------|--------------------------------------------|
| POST    | /follows/:userId           | JWT  | S'abonner / se désabonner d'un utilisateur |
| GET     | /follows/followers/:userId | —    | Liste des abonnés                          |
| GET     | /follows/following/:userId | —    | Liste des abonnements                      |
| POST    | /conversations             | JWT  | Créer ou récupérer une conversation        |
| GET     | /conversations             | JWT  | Conversations de l'utilisateur             |
| GET     | /messages/:conversationId  | JWT  | Messages d'une conversation                |



## 6.7 API externe Last.fm – /api

| Méthode | Route                      | Description                          |
|---------|----------------------------|--------------------------------------|
| GET     | /api/albums/:artist/:album | Détails d'un album                   |
| GET     | /api/artists/:name         | Profil et statistiques d'un artiste  |
| GET     | /api/tracks/:artist/:track | Informations d'une piste             |
| GET     | /api/tags/:tag             | Albums par genre / tag               |
| GET     | /api/search?query=...      | Recherche globale (albums, artistes) |

## 6.8 OAuth – /api/oauth

| Méthode | Route                            | Description                        |
|---------|----------------------------------|------------------------------------|
| GET     | /api/oauth/auth/google           | Démarrer le flux OAuth Google      |
| GET     | /api/oauth/auth/google/callback  | Callback Google → retourne un JWT  |
| GET     | /api/oauth/auth/discord          | Démarrer le flux OAuth Discord     |
| GET     | /api/oauth/auth/discord/callback | Callback Discord → retourne un JWT |

## 7. Authentification & Sécurité

### 7.1 Authentification locale (JWT)

Le système utilise des JSON Web Tokens d'une durée de validité de 24 h. Le payload contient : id, email, username, role.

Flux d'inscription / connexion :

1. Le client envoie ses identifiants à POST /users/signin (inscription) ou POST /users/login (connexion).
2. Le backend valide les données, puis compare le mot de passe via bcrypt.compare() (hash bcrypt, 10 rounds).
3. Un token est signé via jwt.sign() et retourné au client.
4. Le client stocke le token (localStorage côté web, expo-secure-store côté mobile).
5. Toute requête protégée transmet Authorization: Bearer <token> ; le middleware authGuard valide le token via jwt.verify().

Règles de validation (côté serveur) :

- Mot de passe : minimum 8 caractères, avec au moins une majuscule, une minuscule, un chiffre et un caractère spécial.
- Username : 3 à 30 caractères, lettres / chiffres / \_ / - uniquement.
- Email : format vérifié et normalisé (trim + minuscules).

### 7.2 OAuth 2.0 (Google & Discord)

Implémenté via Passport.js avec les stratégies passport-google-oauth20 et passport-discord.

| Étape           | Description                                                              |
|-----------------|--------------------------------------------------------------------------|
| 1. Initiation   | Le client redirige vers /api/oauth/auth/google ou /discord               |
| 2. Consentement | L'utilisateur autorise l'application sur la page du fournisseur          |
| 3. Callback     | Le fournisseur redirige vers /callback avec un code d'autorisation       |
| 4. Traitement   | Passport échange le code contre un profil ; fusion ou création du compte |
| 5. JWT          | Génération d'un JWT standard, puis redirection du client avec le token   |

Fusion de comptes — Si un utilisateur se connecte via OAuth avec un email déjà enregistré localement, le système lie provider et provider\_id au compte existant plutôt que de créer un doublon.

## 7.3 Contrôle d'accès basé sur les rôles (RBAC)

| Rôle  | Permissions                                                                                                                        |
|-------|------------------------------------------------------------------------------------------------------------------------------------|
| BASIC | Accès à toutes les fonctionnalités utilisateur standard (critiques, playlists, messages, profil)                                   |
| ADMIN | Tout BASIC + tableau de bord admin, liste des utilisateurs, suppression de tout contenu, gestion des bannissements et signalements |

## 7.4 Sécurité applicative

- Aucun secret (clé API, secret JWT, identifiants BDD) n'est présent en clair dans le code : tous sont injectés via variables d'environnement.
- Un fichier `.env.exemple` est fourni sans valeurs réelles ; le `.env` est listé dans `.gitignore` et n'est jamais commité.
- Mots de passe hachés avec `bcrypt` (10 rounds).
- Vérification du rôle (BASIC / ADMIN) côté serveur sur chaque route protégée.
- CORS restreint aux origines autorisées.
- Validation et sanitisation des entrées utilisateur côté serveur.
- Protection contre les injections SQL via l'ORM Prisma (requêtes paramétrées).
- Limite de taille des payloads (10 Mo) pour les uploads en base64.

## 8. Temps réel & Notifications

### 8.1 Architecture Socket.io

Le serveur Socket.io est initialisé dans `bin/www.ts` ; la logique métier réside dans `modules/web_socket/socket_server.ts`. Chaque utilisateur authentifié rejoint une room privée (`user:{id}`) en plus des rooms de conversation. À la réception d'un `send_message`, le serveur sauvegarde le message en base, le marque éventuellement comme lu, crée une notification, envoie un push Expo si le destinataire est hors-ligne, puis émet l'événement `receive_message` vers la room concernée.

### 8.2 Notifications push (mobile)

Pour les utilisateurs mobiles non connectés au WebSocket, le serveur utilise l'Expo Server SDK pour envoyer des notifications push via APNs (iOS) et FCM (Android).

- Le token Expo de chaque appareil est enregistré via `POST /users/update-push-token`.
- Les notifications sont émises pour : nouveaux messages, likes, commentaires, nouveaux abonnés.
- Un *throttling* est appliqué pour éviter le spam ; les tokens invalides sont ignorés silencieusement.

## 9. Intégration Last.fm

L'API Last.fm est la source principale de données musicales. Les données récupérées sont mises en cache dans PostgreSQL (table Medias) avec expiration (expires\_at) pour réduire le nombre d'appels externes.

| Module    | Endpoints Last.fm utilisés          | Usage dans Melodia                        |
|-----------|-------------------------------------|-------------------------------------------|
| Albums    | album.getInfo, album.search         | Fiche album, recherche, albums similaires |
| Artistes  | artist.getInfo, artist.getTopTracks | Profil artiste, pistes populaires         |
| Pistes    | track.getInfo, track.getSimilar     | Détail piste, recommandations             |
| Tags      | tag.getTopAlbums                    | Albums par genre (découverte)             |
| Recherche | album.search, artist.search         | Barre de recherche globale                |

Clé API — stockée dans les variables d'environnement du backend. En cas de dépassement de quota, les requêtes échouent silencieusement et le cache existant est retourné si disponible.

Bonus — Algorithme de ranking du feed. Le service feed.ranking.service.ts applique un scoring pondéré aux éléments du fil d'actualité, avec des poids adaptés à chaque type de feed :

| Facteur                           | global | friends | discovery |
|-----------------------------------|--------|---------|-----------|
| Engagement (likes, commentaires)  | 0.4    | 0.2     | 0.0       |
| Fraîcheur (décroît avec le temps) | 0.3    | 0.2     | 0.2       |
| Qualité (note, longueur, titre)   | 0.2    | 0.2     | 0.4       |
| Social (boost si abonnement)      | 0.1    | 0.4     | 0.1       |
| Diversité (pénalité redondance)   | 0.0    | 0.0     | 0.3       |

## 10. Client Web

### 10.1 Architecture React

Le client web est une Single Page Application construite avec React 19 et Vite 7. Le routage est géré par react-router-dom v7. L'accès aux routes protégées est sécurisé par les composants <ProtectedRoute> et <AdminRoute>.

```
clients/web/src/
├── api/      # Fonctions d'appel à l'API (axios)
├── components/ # Composants réutilisables
├── context/   # Contextes React (Auth, Socket)
├── pages/     # Pages correspondant aux routes
├── assets/    # Images, polices, SVGs
└── main.tsx  # Bootstrap React + Router
```

### 10.2 Pages et routing

| Route           | Page           | Protection | Description                              |
|-----------------|----------------|------------|------------------------------------------|
| /               | LandingPage    | Public     | Accueil marketing, liens login/register  |
| /register       | Register       | Public     | Formulaire d'inscription                 |
| /login          | Login          | Public     | Connexion email/password ou OAuth        |
| /auth/callback  | AuthCallback   | Public     | Réception du token après OAuth           |
| /home           | Home           | JWT        | Tableau de bord personnel                |
| /feed           | Feed           | JWT        | Fil d'activité des abonnements           |
| /album/:id      | AlbumDetails   | Public*    | Détail album, critiques, statut          |
| /stats          | Stats          | JWT        | Statistiques personnelles                |
| /library        | Library        | JWT        | Playlists et albums sauvegardés          |
| /notifications  | Notifications  | JWT        | Centre de notifications                  |
| /conversations  | Conversations  | JWT        | Liste et messagerie directe              |
| /profil/:id?    | Profil         | JWT        | Profil public d'un utilisateur           |
| /settings       | Settings       | JWT        | Paramètres du compte                     |
| /admindashboard | AdminDashboard | ADMIN      | Gestion utilisateurs, signalements, bans |

\* La fiche album est accessible sans connexion ; les actions (statut, critique) requièrent une authentification.

## 10.3 Composants principaux

- Layout & navigation : Layout, Header, Sidebar, Footer, ScrollToTop — structure commune des pages authentifiées.
- Sécurité des routes : ProtectedRoute (redirige vers /login sans token), AdminRoute (redirige si rôle ≠ ADMIN), AuthGuard (vérification inline).
- Contenu : AlbumCard, FeedCard, StatCard, UserAvatar — affichage des données musicales et sociales.
- Notifications : NotificationBell — icône du header avec compteur de non-lus, à l'écoute des événements Socket.io en temps réel.

## 11. Application Mobile

### 11.1 Architecture Expo

L'application mobile utilise React Native 0.81 avec Expo 54 et Expo Router 6 (routage basé sur les fichiers). La distribution se fait via EAS.

| Aspect         | Détail                                                        |
|----------------|---------------------------------------------------------------|
| Package ID     | com.biohazardyesorganization.melodia                          |
| Nom affiché    | Melodia                                                       |
| Polices custom | Audiowide, Pacifico, Syncopate (Google Fonts via Expo)        |
| Stockage token | expo-secure-store (chiffré sur l'appareil)                    |
| Notifications  | expo-notifications + Expo Server SDK côté backend             |
| Images         | expo-image-picker avec export base64 (quality 0.5, ratio 1:1) |

### 11.2 Navigation et écrans

Navigation principale en onglets bas (Footer), avec des piles de navigation pour les écrans secondaires.

| Écran        | Fichier               | Description                                   |
|--------------|-----------------------|-----------------------------------------------|
| Accueil      | app/index.tsx         | Onglets (Home, Feed, Library, Stats, Profile) |
| Connexion    | app/login.tsx         | Email/password + OAuth                        |
| Inscription  | app/register.tsx      | Formulaire + onboarding                       |
| Feed         | src/pages/Feed.tsx    | Activités des abonnements avec filtres        |
| Bibliothèque | src/pages/Library.tsx | Playlists personnelles                        |

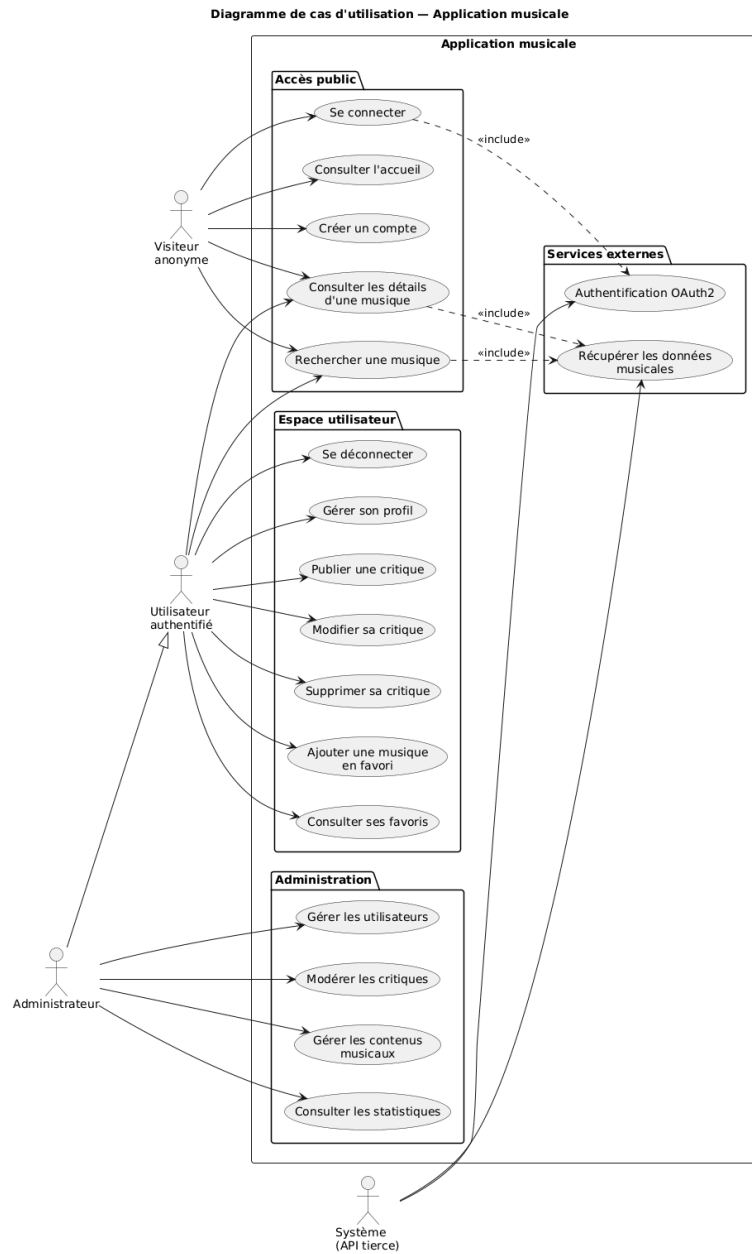
|                 |                                       |                                       |
|-----------------|---------------------------------------|---------------------------------------|
| Statistiques    | src/pages/Stats.tsx + StatDetails.tsx | Compteurs par statut, détails         |
| Profil          | src/pages/Profile.tsx                 | Profil public, critiques, abonnements |
| Modifier profil | src/pages/UpdateProfile.tsx           | Avatar, bio, artiste favori           |
| Détail album    | app/albumdetails.tsx                  | Fiche album, critiques, statut        |
| Écrire critique | app/writereview.tsx                   | Formulaire critique avec note étoiles |
| Commentaires    | app/review/[id]/comments.tsx          | Commentaires threadés                 |
| Conversations   | src/pages/DetailsConversations.tsx    | Messagerie temps réel                 |
| Paramètres      | app/settings.tsx                      | Langue, thème, mot de passe           |
| Admin           | app/admindashboard.tsx                | Dashboard admin (role ADMIN)          |

### 11.3 Système de thèmes (Dark / Light)

Un ThemeContext global gère le basculement entre thème sombre (par défaut) et thème clair. Tous les composants utilisent le hook useTheme() pour accéder aux tokens de couleur. Les propriétés de layout (padding, margin, borderRadius, fontSize) sont définies dans des StyleSheet statiques ; les couleurs sont surchargées inline via la syntaxe array : [styles.text, { color: theme.text }].

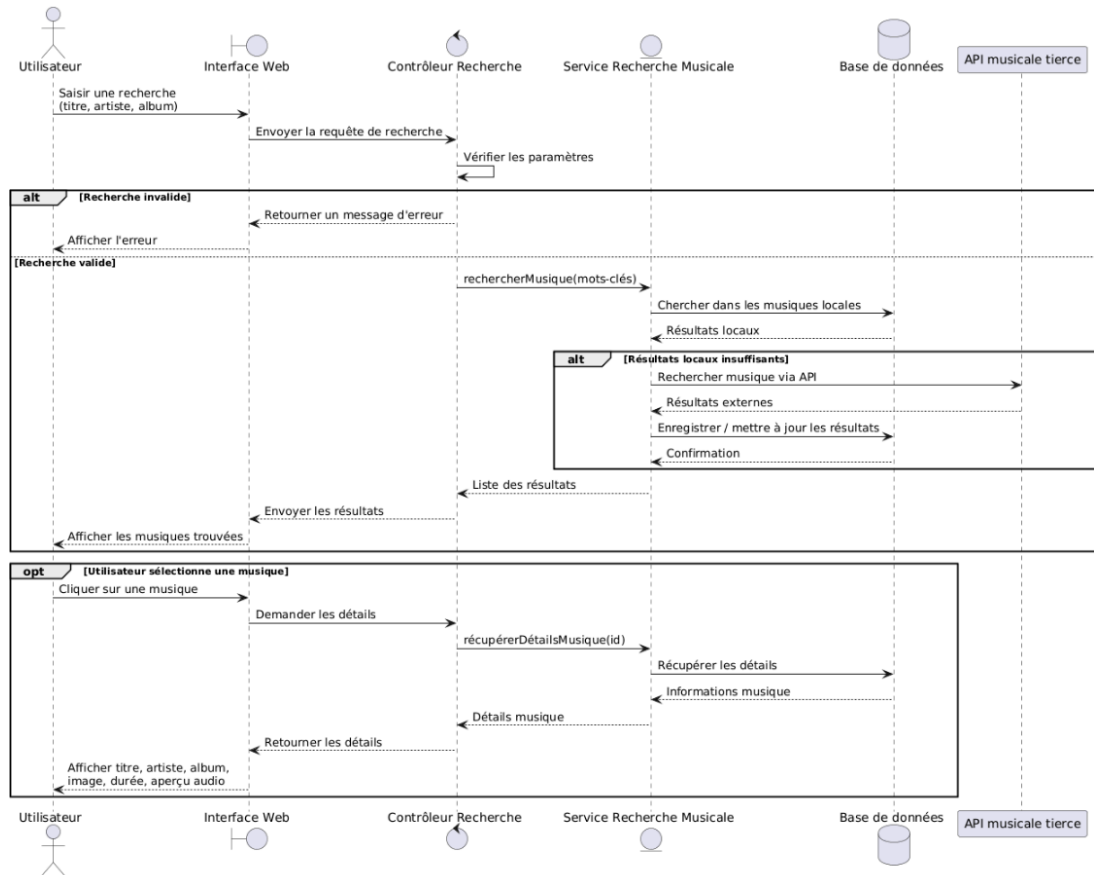
## 12. Diagrammes UML

### 12.1 Diagramme de cas d'utilisation

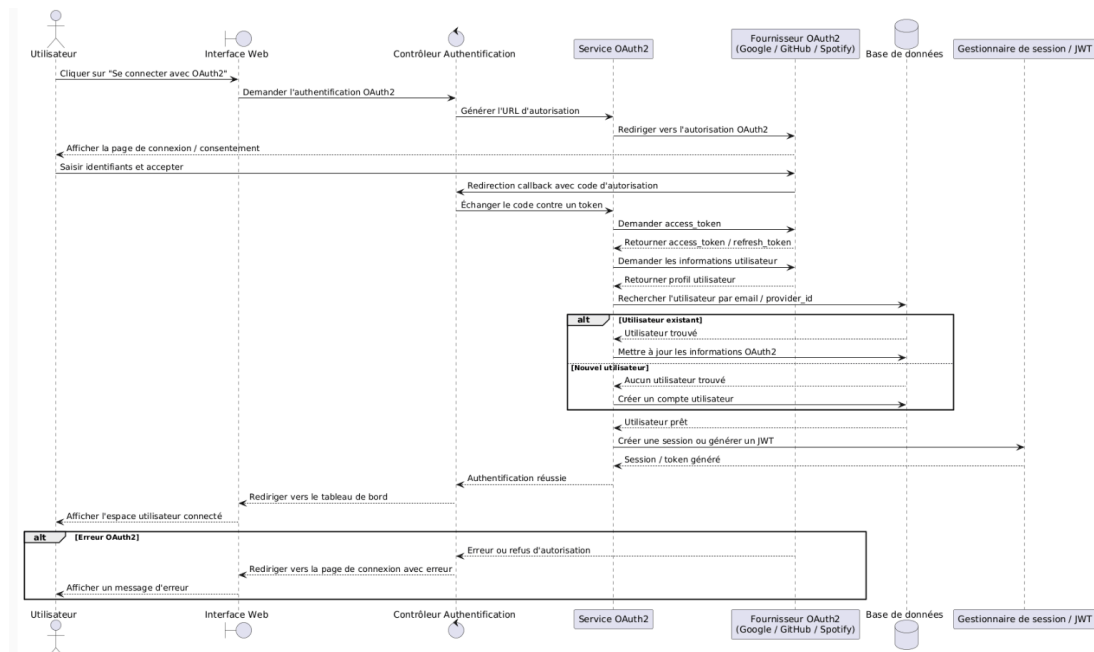




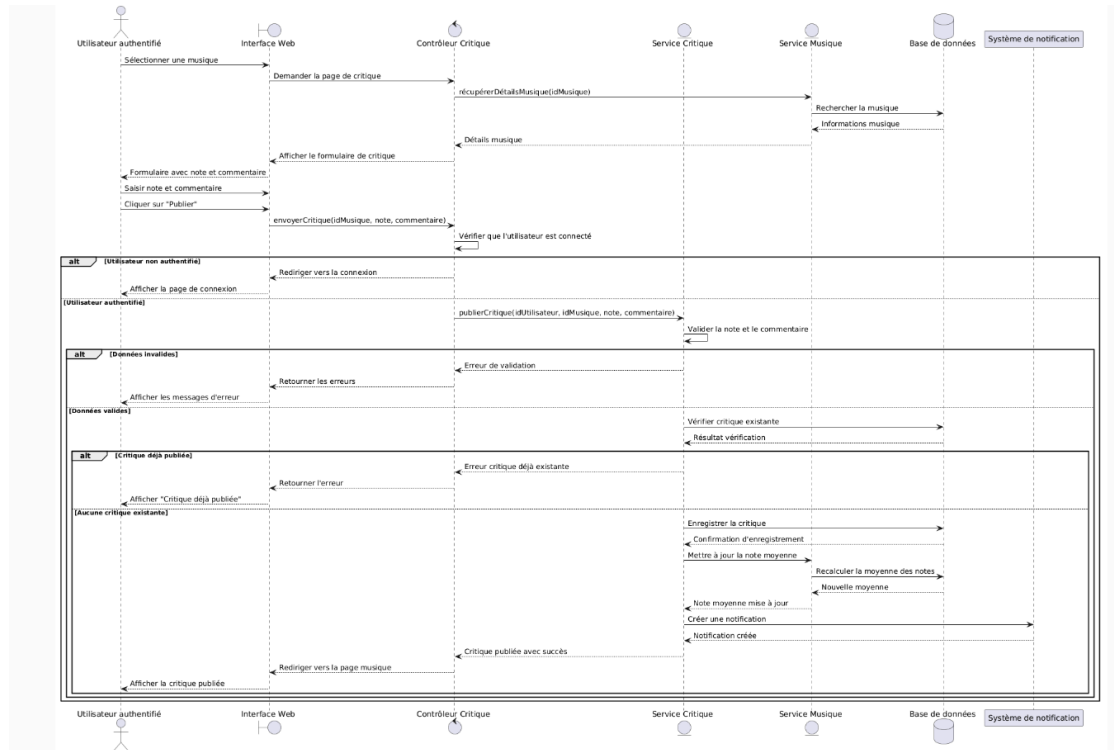
## 12.2 Diagramme de séquence – Recherche musicale



## 12.3 Diagramme de séquence – Authentification OAuth2



## 12.4 Diagramme de séquence – Publication d'une critique



## 13. Infrastructure & Déploiement

### 13.1 Prérequis

| Logiciel              | Version minimale | Lien officiel                                                                 |
|-----------------------|------------------|-------------------------------------------------------------------------------|
| Docker                | 24.x             | <a href="https://docs.docker.com">https://docs.docker.com</a>                 |
| Docker Compose        | 2.x              | <a href="https://docs.docker.com/compose">https://docs.docker.com/compose</a> |
| Git                   | 2.x              | <a href="https://git-scm.com">https://git-scm.com</a>                         |
| Node.js (hors Docker) | 20.x LTS         | <a href="https://nodejs.org">https://nodejs.org</a>                           |
| Expo CLI (mobile)     | dernière         | <a href="https://docs.expo.dev">https://docs.expo.dev</a>                     |

### 13.2 Obtention de la clé API Last.fm

1. Se rendre sur <https://www.last.fm/api/account/create>.
2. Créer un compte Last.fm si nécessaire.
3. Renseigner le formulaire de création d'application.
4. Copier la clé API générée.
5. La coller dans le fichier backend/.env (variable API\_KEY).

### 13.3 Configuration de l'environnement

Copier les fichiers .env.exemple fournis (racine, backend/, clients/web/, clients/mobile/) en .env et renseigner les valeurs.

Backend (backend/.env)

| Variable                                     | Exemple                                                                             | Description              |
|----------------------------------------------|-------------------------------------------------------------------------------------|--------------------------|
| DATABASE_URL                                 | postgresql://user:pass@db:5432/melodia                                              | Connexion PostgreSQL     |
| PORT                                         | 3000                                                                                | Port d'écoute de l'API   |
| JWT_SECRET                                   | un_secret_long_aleatoire                                                            | Clé de signature des JWT |
| API_ROOT_URL                                 | <a href="https://ws.audioscrobbler.com/2.0/">https://ws.audioscrobbler.com/2.0/</a> | URL racine Last.fm       |
| API_KEY                                      | votre_cle_lastfm                                                                    | Clé API Last.fm          |
| GOOGLE_CLIENT_ID /<br>GOOGLE_CLIENT_SECRET   | ...                                                                                 | OAuth Google             |
| GOOGLE_CALLBACK_URL                          | ...                                                                                 | URL de callback Google   |
| DISCORD_CLIENT_ID /<br>DISCORD_CLIENT_SECRET | ...                                                                                 | OAuth Discord            |
| WEB_CLIENT_URL                               | <a href="http://localhost:5173">http://localhost:5173</a>                           | Origine du client web    |

Frontend web (clients/web/.env) — VITE\_API\_URL=<http://localhost:3000>

Mobile (clients/mobile/.env) — EXPO\_PUBLIC\_API\_URL=http://localhost:3000

Racine (.env) — POSTGRES\_DB, POSTGRES\_USER, POSTGRES\_PASSWORD,  
CLOUDFLARE\_TUNNEL\_TOKEN

## 13.4 Lancement avec Docker Compose

```
docker compose up --build -d
```

```
docker compose logs -f
```

| Service              | Port | Accès                     |
|----------------------|------|---------------------------|
| db (PostgreSQL 18.1) | 5432 | Interne Docker            |
| api (Express)        | 3000 | http://localhost:3000     |
| frontend (Nginx)     | 80   | http://localhost          |
| cloudflare-tunnel    | —    | Exposition HTTPS publique |

## 13.5 Architecture Docker Compose

Quatre conteneurs communiquent sur un réseau bridge dédié melodia-network :

- db — PostgreSQL 18.1, volume postgres\_data, variables POSTGRES\_\*.
- api — image melodia-api:latest, build multi-stage Node 20-alpine (tsc → dist/, npx prisma generate), depends\_on: db, point d'entrée node dist/bin/www.js.
- frontend — image melodia-frontend:latest, build multi-stage servi par Nginx (port 80), arg VITE\_API\_URL, mode SPA, depends\_on: api.
- cloudflare-tunnel — cloudflare/cloudflared, exécute le tunnel via CLOUDFLARE\_TUNNEL\_TOKEN.

Les secrets sont injectés via .env et ne figurent jamais dans le fichier Compose.

## 13.6 Domaines et réseau (production)

| Service      | URL                       | Description                   |
|--------------|---------------------------|-------------------------------|
| Frontend web | https://melodia.cloud     | Application principale        |
| Frontend web | https://www.melodia.cloud | Alias www                     |
| API backend  | https://api.melodia.cloud | Endpoints REST + WebSocket    |
| Mobile       | EAS build                 | Binaires iOS/Android via Expo |

Cloudflare Tunnel expose les services internes sans IP publique fixe ni ouverture de ports. Le trafic HTTPS est terminé chez Cloudflare, qui transmet les requêtes aux conteneurs via le tunnel sécurisé.

## 13.7 Commandes de développement

- Backend — npm run dev (nodemon + tsx), npm run build (compile TypeScript), npx prisma migrate dev (migrations), npx prisma studio (interface BDD).
- Web — npm run dev (Vite, :5173), npm run build (bundle production).
- Mobile — npm start (Expo dev server), eas build (build iOS/Android cloud).

## 14. Qualité du code & conventions

### 14.1 Structure des dépôts

Le projet est organisé en monorepo :

```
3PROJ-Team2/
├── backend/      # API REST + WebSocket (Express / Prisma / PostgreSQL)
├── clients/
│   ├── web/     # Application React (Vite + Tailwind)
│   └── mobile/  # Application React Native (Expo)
├── docs/        # Documentation technique
├── docker-compose.yml
└── README.md
```

### 14.2 Conventions de nommage

- Variables et fonctions : camelCase (TypeScript).
- Classes et composants React : PascalCase.
- Constantes : UPPER\_SNAKE\_CASE.
- Fichiers backend : pattern par domaine <entité>.controller.ts, <entité>.service.ts, <entité>.mapper.ts, <entité>.helper.ts.
- Composants / pages frontend : PascalCase.tsx.
- Champs de base de données : snake\_case.

### 14.3 Linting & formatage

Le code TypeScript bénéficie du typage strict du compilateur (tsc). Le formatage suit une indentation cohérente et l'usage systématique des types explicites sur les signatures de fonctions.

### 14.4 Tests

L'application est structurée pour être testable (séparation app.ts / bin/www.ts, services isolés de la couche HTTP). Le mode NODE\_ENV=test désactive les logs d'erreurs pour garder une sortie de test propre.

## 15. Gestion de projet & contribution

### 15.1 Dépôt Git

- Dépôt : <https://github.com/Biohazardyyy/3PROJ-Team2>
- Stratégie de branches : branche main stable, branches de fonctionnalités fusionnées par Pull Request.
- Revue de code : chaque Pull Request est revue par au moins un autre membre de l'équipe.

### 15.2 Répartition des tâches

| Membre          | Rôle principal          | Périmètre                         |
|-----------------|-------------------------|-----------------------------------|
| Noah Guillard   | Lead Backend & Frontend | API, architecture, intégrations   |
| Thibaud Marlier | Lead Backend & Frontend | API, base de données, déploiement |
| Aurélien Berlot | Dev Frontend            | Applications React Native / React |
| Romain Metais   | Dev Frontend            | Applications React Native / React |

## 16. Annexes

### 16.1 Références

- Documentation API Last.fm — <https://www.last.fm/api>
- Documentation Docker — <https://docs.docker.com>
- Express.js — <https://expressjs.com>
- Prisma — <https://www.prisma.io/docs>
- React — <https://react.dev>
- Expo — <https://docs.expo.dev>
- OWASP Top 10 — <https://owasp.org/www-project-top-ten/>

### 16.2 Exemple de réponse API

POST /users/login (succès) :

```
{
  "token": "eyJhbGciOiJIUzI1NiIsInR5cCI6IkpXVCJ9...",
  "user": {
    "id": "9c4d...",
    "username": "melomane",
    "email": "user@example.com",
    "role": "BASIC"
  }
}
```

Erreur normalisée : { "error": 400, "message": "Email or username already in use" }